

Cierre de Sprint – Evidencia Metodológica RUP + SCRUM

Información general del Sprint

- **Sprint:** Sprint No. 2
 - **Periodo:** 10/Feb/2026 – 24/Feb/2026
 - **Objetivo del Sprint:** Finalizar el “Happy Flow” funcional del gameplay, incluyendo las funcionalidades del sistemas de salud, tiempo y una arquitectura base para puzzles que permita la transición entre estados de victoria y derrota. Asimismo, desarrollar e implementar los componentes que califican el desempeño del jugador en los distintos gameplays y crear mecanismos que permitan compartir código entre clases que manejan el PuzzleGameplay de manera eficiente, pero permitiendo que cada clase implemente su código distintivo.
-

Matriz de Trazabilidad

Fase RUP ↔ Objetivos ↔ Actividades ↔ Artefactos

 Tabla de trazabilidad

Fase RUP	Objetivo de la Fase	Actividades realizadas en el Sprint	Artefactos producidos
Incepción	Definir el alcance y		

	requerimientos del juego.		
Elaboración	Definir la arquitectura base de las vidas y el temporizador y mitigar riesgos técnicos de alta dependencia, así como estrechar la arquitectura mediante contratos.	Definición de la interfaz IGameOverSubscriber y métodos abstractos en AbstractPuzzleGameplay; y Diseño y creación de la interfaz IDamageable.	Diagrama de Clases.
Construcción	Desarrollar componentes funcionales y UI. Implementar lógica funcional de juego. Validar el flujo de control (Victory/Defeat).	Creación de PlayerHealthLogic, PlayerHealthView, CountdownBarManager, CountdownTextManager (Separando Logic y View) y lógica de suscripción estática en GameManager. Implementación de funciones Victoria() y Defeat() que permiten la salida del puzzle y retorno al juego base e implementación de IDamageable en la clase del Jugador.	Scripts de Player, UIElementes, IDamageable, GameManager, Prefabs de UI.

Transición	Validar la integración en una escena de prueba.	Verificación del Happy Flow: Entrada al puzzle -> Tutorial -> Juego -> Victoria/Derrota -> Retorno.	Escena con el flujo funcional
------------	---	---	-------------------------------

3 Evidencia de decisiones de diseño por fase

Justificación técnica

En este Sprint se tomaron las siguientes decisiones relevantes:

Decisiones arquitectónicas

- **Decisión:** Template Method Pattern en AbstractPuzzleGameplay
- **Fase RUP asociada:** Elaboración / Construcción.
- **Justificación técnica:** Al definir Victoria() y Defeat() como métodos abstractos, se garantiza que cualquier puzzle nuevo esté obligado a implementar la lógica de victoria que disparará la clase PuzzleManager. Esto asegura que el flujo de "resumir el juego" sea consistente y no dependa de implementaciones manuales en cada puzzle.
- **Impacto en el sistema:** Robustez en el flujo de salida. Evita que el jugador quede "atrapado" en un puzzle por falta de una llamada al método de retorno.

Decisiones arquitectónicas

- **Decisión:** Desacoplamiento de UI mediante Delegates (Delegate predefinido Action)
- **Fase RUP asociada:** Construcción.
- **Justificación técnica:** El uso de delegados para salud y tiempo permite que la lógica del sistema funcione de forma independiente de la implementación de un elemento visual, permitiendo que este último administre cómo se ve la información de manera independiente, pero forzado a implementar un contrato.
- **Impacto en el sistema:** Facilita las pruebas unitarias y permite cambiar el "skin" de la interfaz sin afectar el núcleo matemático del juego.

Decisiones arquitectónicas

- **Decisión:** Implementación de Interfaz de Contrato IDamageable
- **Fase RUP asociada:** Elaboración / Construcción.
- **Justificación técnica:** Se elimina la dependencia directa entre los "Damagers" (trampas, enemigos, puzzles) y la clase específica del Jugador. Al utilizar una interfaz, cualquier clase que implemente TakeDamage(int amount) o algún método similar sobrecargado puede ser procesada por el mismo sistema de ataque.
- **Impacto en el sistema:** Polimorfismo puro. Permite que el sistema sea extensible (Open-Closed Principle); podemos añadir 100 tipos de objetos destructibles sin cambiar una sola línea de código en el script que infringe el daño.

Organización del sistema

- **Separación aplicada:**
 - Core
 - Sistema de Puzzles
 - Persistencia
- **Motivo:** d
- **Beneficio esperado:** d

Patrones o principios utilizados

- **Patrón / Principio:** Template Method (en la Clase Abstracta) / Observer (vía Delegates estáticos) / DRY (Don't Repeat Yourself) / SoC (Separation of Concerns) / Interface Segregation (ISP) / Dependency Inversion (DIP)
- **Contexto de uso:**
 - Template Method -> La clase AbstractPuzzleGameplay define el esqueleto de cómo se gana o se pierde, pero deja que cada puzzle específico (p. ej., un puzzle de cables o uno de memoria) decida cuándo y cómo ocurre esa victoria.
 - Observer -> OnGameOver en el GameManager. Al ser estático, permite una suscripción global inmediata, resolviendo el problema de las dependencias cruzadas en el ciclo de vida de Unity (Awake/Start).
 - DRY -> Centralización de referencias (DataManager, BackgroundPrefab) en la clase abstracta de los puzzles para evitar duplicar código en cada nuevo puzzle creado.
 - SoC -> El TimerManager (Lógica) solo calcula segundos; el TextManager (Vista) solo formatea strings. Asimismo con el countdownBar (Lógica) y el CountdownText (Vista)

- ISP / DIP -> Dependency Inversion: Los sistemas que causan daño ahora dependen de la abstracción IDamageable en lugar de la implementación concreta en PlayerHealthLogic.
Polimorfismo: Facilita que un "Raycast" o un "Trigger" de daño simplemente busque el componente de interfaz, simplificando la lógica de colisiones

- **Fase RUP:** Elaboración / Construcción
-

4 Bitácora resumida por fase RUP

Incepción

Qué se hizo:

Se revisó el alcance del sprint y los requerimientos necesarios para completar el flujo básico del gameplay del puzzle, identificando los sistemas principales que debían implementarse: salud del jugador, temporizador y condiciones de victoria o derrota.

Por qué se hizo:

Se realizó para asegurar que el sprint estuviera alineado con el objetivo de lograr un flujo funcional completo del minijuego y definir claramente qué componentes debían desarrollarse.

Resultado:

Objetivos del sprint definidos y claridad sobre los sistemas base necesarios para soportar el gameplay del puzzle.

Elaboración

Qué se hizo:

Se diseñó la arquitectura base del sistema de puzzles y de los sistemas de salud y tiempo. También se definieron contratos mediante interfaces como IGameOverSubscriber e IDamageable, así como la estructura de la clase abstracta AbstractPuzzleGameplay.

Por qué se hizo:

Se buscó reducir riesgos técnicos y establecer una arquitectura flexible que permitiera reutilizar código entre distintos puzzles manteniendo la capacidad de personalizar su lógica.

Resultado:

Base arquitectónica definida mediante interfaces, clases abstractas y diagramas de clase que estructuran el funcionamiento del sistema.

Construcción

Qué se hizo:

Se implementaron los sistemas funcionales del juego, incluyendo la lógica y vista del sistema de salud (PlayerHealthLogic y PlayerHealthView), el temporizador (CountdownBarManager y CountdownTextManager), así como la lógica de victoria y derrota dentro del flujo del puzzle.

Por qué se hizo:

Se desarrollaron estos componentes para materializar la arquitectura diseñada y permitir que el jugador interactúe con el puzzle dentro de un flujo completo de juego.

Resultado:

Prototipo funcional del gameplay con sistemas de salud, temporizador y transición entre estados de victoria o derrota.

Transición

Qué se hizo:

Se realizaron pruebas de integración en una escena de prueba verificando el flujo completo del juego: entrada al puzzle, tutorial, desarrollo del gameplay y retorno al juego principal tras ganar o perder.

Por qué se hizo:

Se evaluó el funcionamiento conjunto de los sistemas implementados para validar que el flujo del jugador fuera consistente y que las transiciones entre estados se ejecutaran correctamente.

Resultado:

Comprobación del flujo funcional del puzzle dentro de una escena de prueba, confirmando la correcta integración de los sistemas desarrollados.

5 Validación del incremento (si aplica)

- **Funcionalidad:** <OK / Observaciones>
- **Fiabilidad:** <OK / Observaciones>
- **Rendimiento:** <OK / Observaciones>
- **Usabilidad:** <OK / Observaciones>